

Vietnam General Confederation of Labor
Ton Duc Thang University
Faculty of Information Technology



Dinh Quoc Cuong – 523H0008
Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT
DEVOPS, CONTAINERIZATION & CLOUD
SYSTEMS

Supervisor
Msc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, we would like to express our sincere gratitude to our supervisor, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and the academic rigour of this work.

We are also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for our work. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to our growth.

Our gratitude further extends to our classmates and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, we are profoundly grateful to our families for their unconditional support, patience, and encouragement. Their belief in us has always been our greatest source of strength and determination.

To all of you, we extend our heartfelt thanks.

Certification

This thesis was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(*Title, Full Name and Signature*)

This thesis was defended at the Undergraduate Thesis Examination Committee held
at Ton Duc Thang University on / /

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee
and the Dean of the Faculty after receiving the revised thesis (if any).

CHAIRMAN

DEAN OF FACULTY

.....

Declaration

This thesis was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this thesis are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this thesis. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this thesis. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

Author(s)

(Signature and full name)

Abstract

Modern video sharing platforms must solve a common core problem: when a user uploads a source file in an arbitrary format, codec, or resolution, the system has to transcode it automatically into HTTP Live Streaming (HLS) so that browsers can play it with adaptive quality. Provisioning a dedicated server cluster or a Kubernetes system that runs continuously purely to wait for transcoding work is wasteful, because upload traffic is inherently bursty and the infrastructure sits idle most of the time. Conversely, when many users upload simultaneously, the system must scale out to process hundreds of videos in parallel.

This project presents the design, implementation, and evaluation of a complete video sharing platform whose transcoding subsystem is built on a **Serverless Container** and **Event-Driven** architecture. Uploading a file to Amazon S3 emits an event that is buffered in Amazon SQS and dispatched by an AWS Lambda job submitter to AWS Batch running on AWS Fargate. A container packaging FFmpeg is created only when there is work to do, produces three renditions at 360p, 720p, and 1080p together with a master playlist and a thumbnail, uploads the result to a processed bucket, updates the metadata in MongoDB Atlas, and then terminates. Content is delivered through Amazon CloudFront with Origin Access Control, while private videos are additionally protected by CloudFront Signed Cookies so that unauthorised viewers are rejected at the edge even when they know the exact manifest URL.

The entire AWS infrastructure is described as code using eleven independent Terraform modules and can be provisioned with a single command. Seven GitHub Actions workflows automate linting, unit testing, static security analysis, container image scanning, infrastructure validation, and deployment, with a two-tier quality gate that blocks a pull request whenever a fixable critical vulnerability is detected.

The system is evaluated along four dimensions: quality of experience measured through Time-to-First-Frame, transcoding pipeline latency across 100 MB, 500 MB, and 1 GB inputs, scalability under a concurrent upload stress test, and operating cost compared against a traditional always-on Amazon EC2 deployment. The cost analysis shows that, for the architecture-dependent portion of the bill, the serverless approach removes almost all idle resource waste, although this advantage narrows as sustained utilisation increases.

List of Abbreviations

ABR	Adaptive Bitrate Streaming
ACM	AWS Certificate Manager
API	Application Programming Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
DLQ	Dead Letter Queue
DRM	Digital Rights Management
ECR	Amazon Elastic Container Registry
ERD	Entity Relationship Diagram
FinOps	Cloud Financial Operations
HLS	HTTP Live Streaming
IaC	Infrastructure as Code
IAM	AWS Identity and Access Management
JWT	JSON Web Token
OAC	Origin Access Control
QoE	Quality of Experience
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SPA	Single Page Application
SQS	Amazon Simple Queue Service
S3	Amazon Simple Storage Service
TTFB	Time-to-First-Frame
TTL	Time To Live
VMAF	Video Multi-Method Assessment Fusion
VOD	Video on Demand
VPC	Amazon Virtual Private Cloud

Contents

Acknowledgment	i
Certification	ii
Declaration	iii
Abstract	iv
List of Abbreviations	v
1 Introduction	1
1.1 Background and Rationale	1
1.2 Project Objectives	2
1.3 Scope and Deliverables	2
1.4 Main Contributions	2
1.5 Structure of the Report	3
2 Literature Review and Technologies	4
2.1 Related Platforms and Comparison	4
2.2 Core Models and Principles	5
2.2.1 Serverless Containers	5
2.2.2 Event-Driven Architecture	6
2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder	6
2.3 Technology Stack Justification	6
3 Requirements Analysis	8
3.1 Stakeholders and User Requirements	8
3.2 Functional Requirements	8
3.3 Non-Functional Requirements	9
3.4 Use Case Model	10
4 System Design	13
4.1 High-Level Architecture	13

4.2	Transcoding Pipeline Design	15
4.3	Database Design	16
4.4	Content Protection Design	18
4.5	Interface Design	19
5	Implementation and Experimental Results	20
5.1	Backend Implementation	20
5.2	Transcoder Implementation	21
5.3	Infrastructure as Code	22
5.4	CI/CD Pipeline and DevSecOps	22
5.5	Testing	24
5.6	Quality of Experience Evaluation	25
5.7	Transcoding Pipeline Performance	25
5.8	Load Testing	27
5.8.1	Cross-Validation with an Independent Tool	28
5.9	Cost Analysis	30
6	Conclusion and Future Work	31
6.1	Summary of Achievements	31
6.2	Limitations	32
6.3	Future Work	33
A	Supplementary Material	37
A.1	Repository Structure	37
A.2	FFmpeg Transcoding Command	37
A.3	Signed Cookie Custom Policy	38
A.4	Reproducing the Measurements	38
A.5	Deploying the Infrastructure	38

List of Figures

3.1	<i>Use case diagram of the video sharing platform</i>	11
4.1	<i>Overall system architecture and the eleven-step data flow</i>	14
4.2	<i>Sequence diagram of the event-driven transcoding pipeline</i>	15
4.3	<i>Entity relationship diagram of the MongoDB data model</i>	17
5.1	<i>CI/CD workflows and the two-tier DevSecOps quality gate</i>	23

List of Tables

2.1	Comparison of transcoding approaches across platforms	5
3.1	Functional requirements of the system	9
3.2	Non-functional requirements of the system	10
3.3	Use case specification: upload a video	12
5.1	Terraform modules and their responsibilities	22
5.2	Automated test coverage by area	24
5.3	Time-to-First-Frame measurement results	25
5.4	Transcoding pipeline latency by input size	26
5.5	Concurrent upload stress test results (API layer)	28
5.6	Cross-validation of the load test across two independent tools	28
5.7	Monthly operating cost comparison (ap-southeast-1)	30
A.1	Measurement scripts and their output artefacts	38

Chapter 1

Introduction

1.1 Background and Rationale

Video has become the dominant form of content on the Internet, and platforms such as YouTube, Vimeo, and TikTok have made uploading and sharing video an everyday activity. Behind that apparent simplicity lies a demanding engineering problem. When a user uploads a source file, that file may use any container format, any codec, and any resolution, yet the platform must deliver a stream that plays smoothly on every device and adapts to whatever bandwidth the viewer happens to have at that moment.

The industry solution to this problem is HTTP Live Streaming (HLS), a protocol that splits a video into short segments, typically between two and ten seconds long, and produces several quality renditions of the same content. A playlist file written in the `.m3u8` format enumerates the available renditions, and the player selects among them in real time according to measured network throughput. This mechanism, known as Adaptive Bitrate Streaming (ABR), is what allows playback to degrade gracefully rather than stall when a connection weakens.

Producing those renditions is computationally expensive, and this is where the architectural difficulty arises. Transcoding is a bursty workload: it consumes a large amount of CPU while a video is being processed and nothing at all when no one is uploading. A platform that keeps a dedicated server cluster or a Kubernetes system running continuously simply to wait for work therefore pays for a substantial amount of idle capacity. The alternative failure mode is equally undesirable: a system provisioned only for the average case cannot absorb a sudden burst when many users upload at the same time, and uploads either queue for a long time or are dropped altogether.

This tension between idle cost and burst capacity motivates the present project. Rather than maintaining permanently allocated compute, the system proposed here starts a container only when there is a video to process and destroys it immediately afterwards, so that cost accrues only for the seconds during which transcoding actually runs.

1.2 Project Objectives

The project aims to build a complete video sharing platform for end users, together with the automated transcoding subsystem that supports it. Specifically, the objectives are:

1. To build a web platform that allows users to register an account, upload videos, watch them at multiple quality levels, and share them through either public or private links.
2. To automate the entire processing pipeline, from the moment a source file lands in Amazon S3 through transcoding, storage, and delivery via a content delivery network, until the video becomes ready for playback in a browser.
3. To perform transcoding with FFmpeg inside a Docker container running on AWS Batch and AWS Fargate, such that containers are created on demand and terminate automatically, thereby eliminating the cost of idle resources.
4. To automate software deployment and infrastructure management through a CI/CD pipeline and Infrastructure as Code.
5. To evaluate the resulting system empirically in terms of quality of experience, transcoding performance, scalability under load, and operating cost.

1.3 Scope and Deliverables

The scope of this project covers the full vertical slice of a video sharing service: the browser-based user interface, the backend API, the transcoding engine, the cloud infrastructure that hosts them, and the automation that deploys them.

Several areas are deliberately placed outside the scope. The project does not implement digital rights management systems such as Widevine or FairPlay, which would be required to prevent an authorised viewer from redistributing content they can legitimately access. It does not implement live streaming, since the architecture is designed around video on demand. It also does not attempt per-title bitrate ladder optimisation, using instead a fixed ladder of three renditions; the rationale and the trade-off involved are discussed in Chapter 6.

1.4 Main Contributions

The principal contribution of this work is an end-to-end demonstration that a serverless container architecture can serve as the transcoding backbone of a video platform, together with an empirical assessment of what that choice costs and what it saves.

More concretely, the work contributes an event-driven pipeline in which an upload event propagates through Amazon S3, Amazon SQS, and AWS Lambda to AWS Batch without any polling service in between; a containerised FFmpeg worker that implements the heartbeat pattern to extend the SQS visibility timeout during long transcoding jobs, thereby preventing duplicate processing; a two-layer content protection scheme combining Origin Access Control with CloudFront Signed Cookies so that private videos remain inaccessible even to a viewer holding the exact manifest URL; a complete Infrastructure as Code description of the AWS environment in eleven Terraform modules; and a DevSec-Ops pipeline whose security gate genuinely blocks merges rather than merely reporting findings.

1.5 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 reviews how comparable platforms solve the transcoding problem and justifies the technology choices made here. Chapter 3 analyses the functional and non-functional requirements of the system and presents the use case model. Chapter 4 describes the architecture, the data model, and the interaction design. Chapter 5 details the implementation of each subsystem together with the infrastructure automation and the experimental results. Chapter 6 summarises what was achieved, states the limitations honestly, and outlines directions for future work.

Chapter 2

Literature Review and Technologies

2.1 Related Platforms and Comparison

Understanding how established platforms solve the transcoding problem provides the necessary context for the design decisions taken in this project.

Netflix offers the most thoroughly documented case. Its original video processing system, internally known as Reloaded, was a monolithic pipeline that proved difficult to evolve as requirements changed. Netflix subsequently rebuilt it as a set of independent microservices running on a platform called Cosmos, decomposing the workload into a Video Inspection Service, a Video Encoding Service, and a Video Quality Service [1]. Cosmos itself is layered: an API tier receives requests, a workflow tier orchestrates the processing steps, and a serverless compute tier performs the actual media work, with the three tiers communicating asynchronously through a priority-based messaging system [2]. Two aspects of this design are directly relevant here. First, Netflix also relies on asynchronous messaging rather than synchronous calls to decouple job submission from job execution, which is precisely the role played by Amazon SQS in this project. Second, Netflix moved from linear encoding to distributed chunk-based encoding, allowing hundreds of machines to encode segments of the same title in parallel and reducing processing latency from days to hours.

Amazon Web Services publishes a reference architecture for video on demand that provides a useful point of comparison at the level of managed services [3]. That architecture routes an upload from Amazon S3 through an AWS Lambda job submitter into AWS Elemental MediaConvert, stores the output in a second S3 bucket, and delivers it through Amazon CloudFront, with AWS Step Functions orchestrating the workflow. The architecture presented in this report follows the same overall shape but substitutes a self-packaged FFmpeg container on AWS Batch for MediaConvert. This substitution is the central engineering trade-off of the project: MediaConvert is a fully managed service billed per minute of output and requires almost no operational effort, whereas a self-managed

container is considerably cheaper per unit of work but transfers the responsibility for image maintenance, dependency updates, and failure handling onto the development team. AWS documentation also confirms that Origin Access Control is the currently recommended pattern for securing an S3 origin, superseding the legacy Origin Access Identity mechanism, which validates the approach adopted in Section 4.4.

Table 2.1 summarises the comparison across the dimensions most relevant to this work.

Table 2.1: Comparison of transcoding approaches across platforms

ASPECT	NETFLIX (COSMOS)	AWS VOD REFERENCE	THIS PROJECT
Compute model	Dedicated microservice platform with serverless compute tier	Managed service (MediaConvert)	Serverless container on AWS Batch / Fargate
Parallelism	Chunk-based, distributed across many nodes	Managed internally by the service	One container per video
Bitrate ladder	Per-shot optimisation driven by VMAF	Configurable job template	Fixed ladder: 360p, 720p, 1080p
Queueing	Priority-based messaging system	AWS Step Functions	Amazon SQS with dead letter queue
Content protection	DRM	Signed URLs or signed cookies	OAC plus CloudFront Signed Cookies
Operational effort	Very high	Very low	Moderate

2.2 Core Models and Principles

2.2.1 Serverless Containers

The term serverless container describes a model in which a container image is executed without the developer provisioning or managing the underlying virtual machines, and in which billing is based on execution time rather than allocated capacity. AWS Fargate is the implementation used in this project.

The choice of Fargate over a function-as-a-service platform such as AWS Lambda deserves justification, because Lambda is the more common answer to serverless workloads. Video transcoding is unsuitable for Lambda in this context for two reasons. The first is duration: transcoding a large file routinely exceeds the maximum execution time of a Lambda invocation. The second concerns resource configuration. A measurement study of serverless video processing found that selecting a cost-efficient memory configuration for transcoding is far from trivial, because the optimal setting depends not only on the type of task but even on the content of the video itself, and the cost difference between the least and most generous memory configuration reached 498% for sequential workloads [4].

Fargate avoids this difficulty by allowing vCPU and memory to be specified explicitly and independently.

A known weakness of the container model is cold start latency. A systematic review of the subject reports that more than half of function invocations spend at least 80% of their startup time pulling the container image [5]. This finding directly motivates the use of a multi-stage Docker build in this project, described in Section 5.2, whose purpose is to minimise the size of the image that must be transferred before work can begin.

2.2.2 Event-Driven Architecture

An event-driven architecture propagates work through the emission and consumption of events rather than through direct synchronous invocation. In this project, the completion of an upload to Amazon S3 emits an event notification that is placed on an Amazon SQS queue, from which an AWS Lambda function submits a job to AWS Batch.

The queue is not merely a routing mechanism; it is the component that makes burst absorption possible. If a hundred users upload simultaneously, the queue retains all hundred messages while AWS Batch works through them at whatever rate the compute environment allows. Without this buffer, the system would have to either reject work or provision for peak load permanently. The queue is paired with a dead letter queue so that messages which repeatedly fail processing are isolated for inspection rather than being retried indefinitely.

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder

HLS delivers a video as a master playlist referencing several rendition playlists, each of which lists the media segments for one quality level. The player measures throughput and switches between renditions at segment boundaries.

The set of resolution and bitrate pairs offered is called the bitrate ladder, and its construction is an active research area. Netflix demonstrated that the optimal ladder is not fixed but content-dependent, and that the ideal operating points lie on the convex hull of the quality-versus-bitrate curve [6]. By encoding each shot separately under the control of the VMAF perceptual quality metric, Netflix reported bitrate reductions of between 28% and 38% at equivalent visual quality [7]. The present project uses a fixed three-rung ladder, which is simpler to implement and reason about; the implications of this simplification are revisited in Section 6.3.

2.3 Technology Stack Justification

The frontend is implemented as a React single page application built with Vite, using HLS.js for playback. HLS.js is necessary because most desktop browsers do not support

HLS natively; it parses the manifest in JavaScript and feeds segments to the browser through the Media Source Extensions API. Safari, which does support HLS natively, is handled through a fallback path.

The backend is built on Node.js with Express. The decisive factor is that the backend's workload is almost entirely input/output bound rather than CPU bound: it issues pre-signed URLs, queries MongoDB, and returns JSON, while the heavy computation is deliberately delegated to the transcoder containers. Node.js's event loop is well suited to this profile.

MongoDB Atlas provides the persistence layer. Video metadata is naturally document-shaped and its schema evolves as features are added, which suits a document store better than a rigid relational schema. Using the managed Atlas service also removes the need to operate a database cluster inside the project's own infrastructure.

Terraform was selected for Infrastructure as Code over alternatives such as AWS CloudFormation because its module system allows the infrastructure to be decomposed into reusable units with explicit inputs and outputs, and because its plan step makes the consequences of a change visible before it is applied. GitHub Actions was selected for CI/CD because the source code is already hosted on GitHub, which removes the need for a separate automation server.

Chapter 3

Requirements Analysis

3.1 Stakeholders and User Requirements

Three groups interact with the system, and their differing needs shape the requirements.

The **anonymous visitor** arrives without an account. This group needs to browse public videos, search and filter by category, and watch content immediately, because requiring registration before any content can be seen would deter adoption. The **registered user** additionally needs to upload videos, control whether each video is public or private, manage the videos on their own channel, and interact through likes and comments. The **system administrator**, in the context of this project the development team itself, needs the platform to be observable and deployable without manual intervention.

3.2 Functional Requirements

Table 3.1 enumerates the functional requirements derived from the stakeholder analysis. Each requirement is given an identifier so that it can be traced through the design and implementation chapters.

Table 3.1: Functional requirements of the system

ID	REQUIREMENT	DESCRIPTION
FR-01	Registration and login	Users register with a username, email address, and password. Authentication uses JSON Web Tokens carried in the Authorization header.
FR-02	Password recovery	Users request a reset link by email; the token is single-use and expires after fifteen minutes.
FR-03	Direct upload to S3	The client requests a pre-signed URL from the backend and transfers the file directly to Amazon S3 without routing the payload through the application server.
FR-04	Automatic transcoding	Once the upload completes, the system transcodes the source into HLS renditions without any manual trigger.
FR-05	Adaptive playback	The player switches automatically among 360p, 720p, and 1080p according to available bandwidth.
FR-06	Visibility control	Each video is public or private, and the owner can change this setting after upload.
FR-07	Channel management	Users view their own uploads, including those still processing, and may delete a video together with all of its stored artefacts.
FR-08	Search and filtering	Users search by title, description, or tags, and filter by category.
FR-09	Engagement	Users express likes and dislikes and post comments; a comment may be removed by its author or by the video owner.
FR-10	View counting	The system records a view only after playback has genuinely started, and does not count repeated views from the same viewer within a short window.

Requirement FR-03 deserves particular comment because it shapes the architecture more than any other functional requirement. Routing a multi-gigabyte upload through the backend would consume server memory and bandwidth proportional to the number of concurrent uploads and would make the API server the bottleneck of the entire platform. Issuing a time-limited pre-signed URL and letting the browser transfer the file directly to Amazon S3 removes that bottleneck entirely: the backend handles only a small JSON request regardless of file size.

3.3 Non-Functional Requirements

The non-functional requirements are stated in measurable terms wherever possible, so that Chapter 5 can report whether they were met.

Table 3.2: Non-functional requirements of the system

ID	ATTRIBUTE	REQUIREMENT
NFR-01	Latency	Time-to-First-Frame should remain low enough that playback feels immediate; the measurement methodology is defined in Section 5.6.
NFR-02	Scalability	The pipeline must absorb a burst of concurrent uploads without losing work, processing them as capacity permits rather than rejecting them.
NFR-03	Reliability	A failed transcoding job must not silently disappear; failures are retried and, after repeated failure, isolated in a dead letter queue.
NFR-04	Cost efficiency	Compute cost must accrue only during actual transcoding, with no charge for idle capacity.
NFR-05	Security of content	Private videos must be inaccessible to unauthorised viewers even when the exact manifest URL is known.
NFR-06	Security of the API	Authentication endpoints must resist brute-force attempts, and the origin of cross-origin requests must be restricted to an explicit allow-list.
NFR-07	Reproducibility	The entire infrastructure must be reproducible from source control through a single command.
NFR-08	Maintainability	Changes must pass automated tests and security scanning before reaching the main branch.

Requirement NFR-05 is worth distinguishing from ordinary access control. Restricting an API response is straightforward, but a video is not delivered by the API: it is delivered by a content delivery network as hundreds of individual segment files. An access control scheme that protects only the API therefore leaves the actual content unprotected, a gap that Section 4.4 addresses explicitly.

3.4 Use Case Model

Figure 3.1 presents the use case diagram of the system. Three actors appear: the anonymous visitor, the authenticated user, and the transcoding system itself, which is modelled as an actor because it initiates processing autonomously in response to events rather than in response to a direct user action.

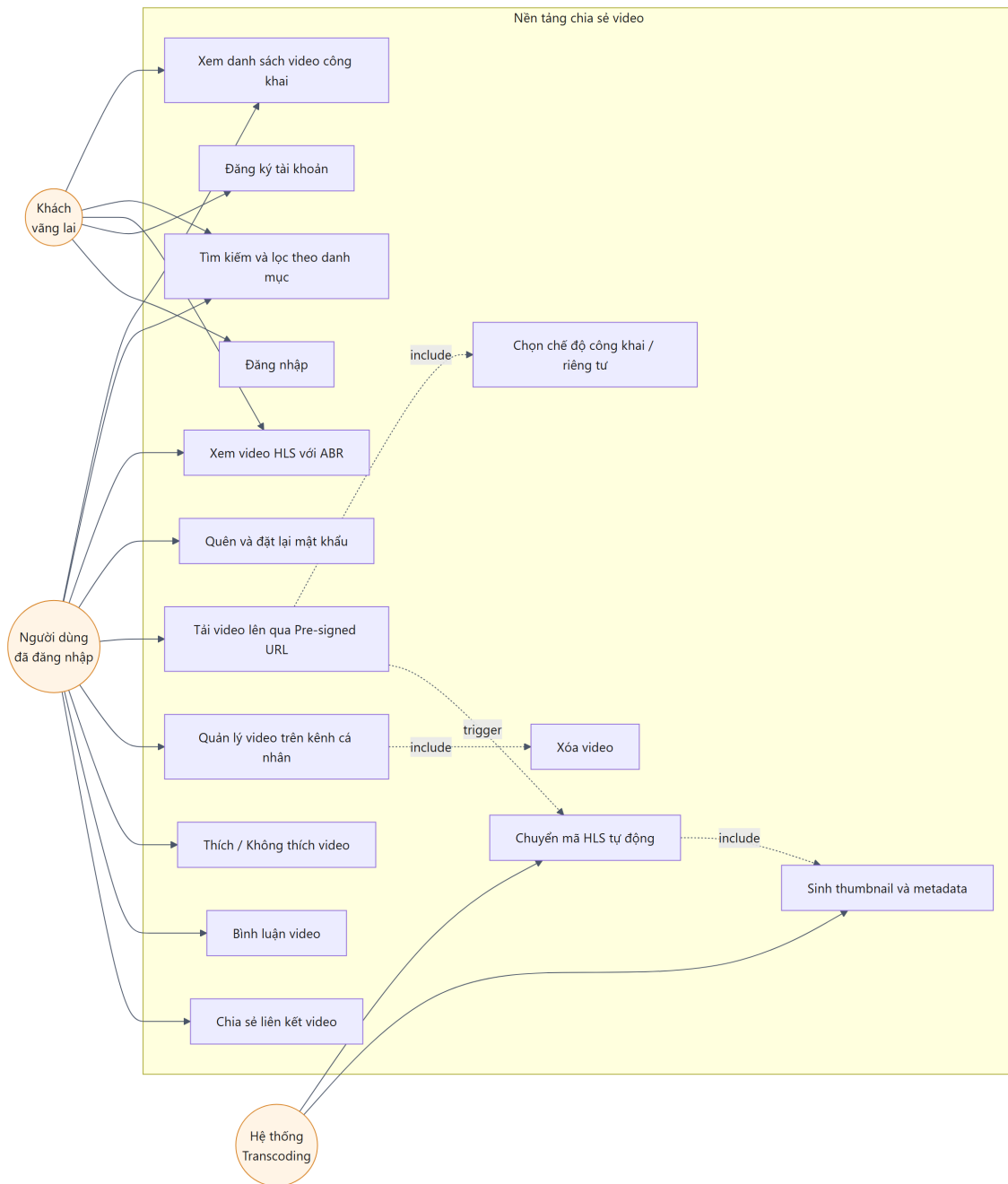


Figure 3.1: *Use case diagram of the video sharing platform*

The diagram makes visible an important property of the design: the transcoding use cases are not invoked by the user at all. The user's upload action triggers them indirectly through the event chain, which is why they are attached to the system actor rather than to the human actors. This separation is what allows the browser to return control to the user immediately after the upload completes, without waiting for processing to finish.

Table 3.3 specifies the most architecturally significant use case in detail.

Table 3.3: Use case specification: upload a video

FIELD	CONTENT
Identifier	UC-07
Actor	Authenticated user
Precondition	The user holds a valid JSON Web Token.
Trigger	The user selects a video file and submits the upload form.
Main flow	1. The client validates the file type and size locally. 2. The client calls the initiate-upload endpoint with the file metadata. 3. The server validates the metadata, creates a database record with status UPLOADING, and returns a pre-signed URL valid for fifteen minutes. 4. The browser transfers the file directly to the raw S3 bucket. 5. The client confirms completion and the record moves to PROCESSING. 6. The transcoding pipeline runs asynchronously and eventually sets the record to READY.
Alternative flow	If the file type is not an accepted video format or the size exceeds the limit, the server rejects the request before issuing a URL, and no orphan record is created.
Postcondition	The video appears on the user's channel, initially as processing and subsequently as ready to play.

Chapter 4

System Design

4.1 High-Level Architecture

The system is organised into four tiers, shown in Figure 4.1: a client tier running in the browser, a delivery tier built on Amazon CloudFront, an application tier comprising the backend API and the database, and a processing tier that performs transcoding.

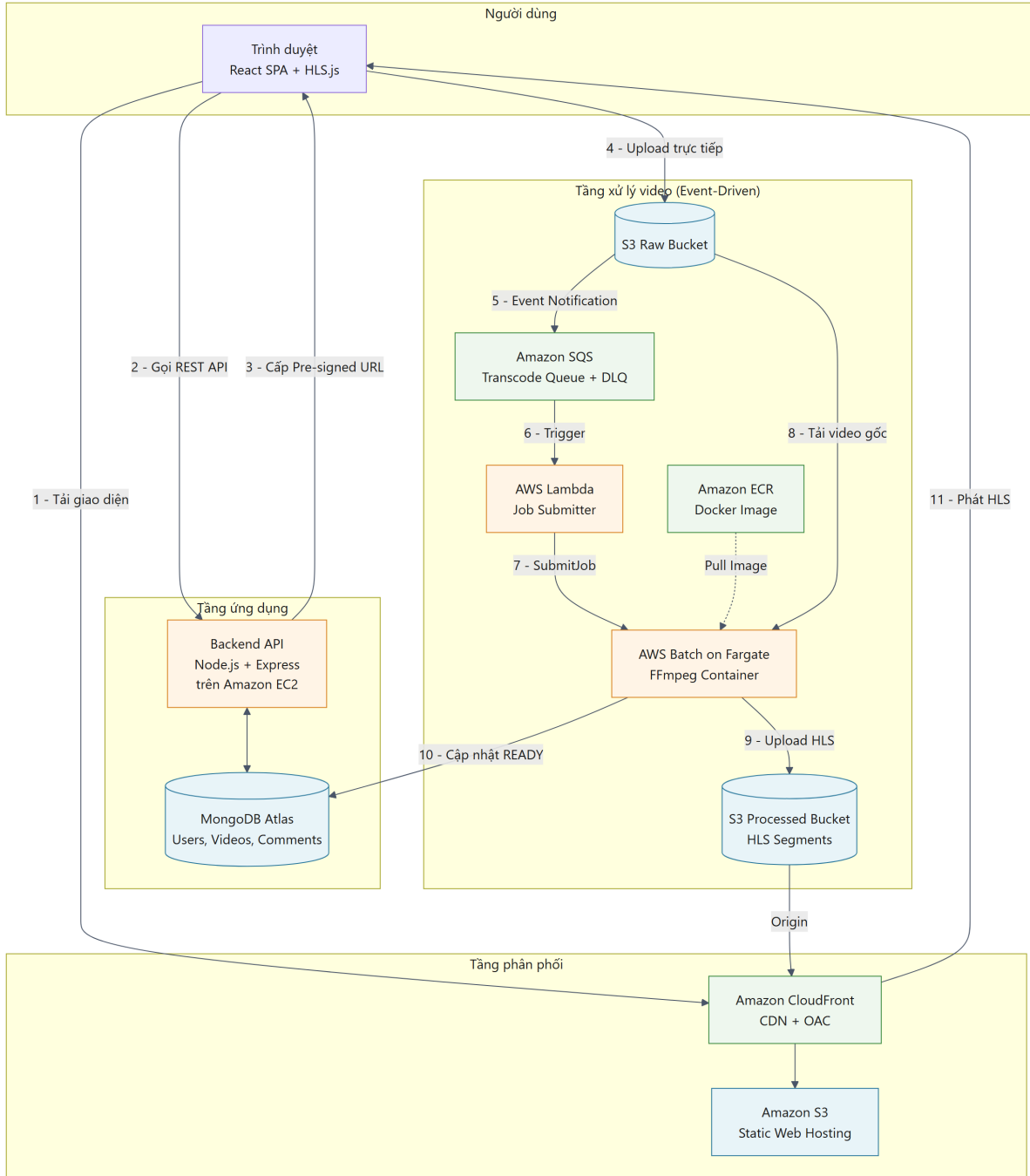


Figure 4.1: Overall system architecture and the eleven-step data flow

The separation between the application tier and the processing tier is the defining characteristic of this design. The backend API never touches video content: it issues pre-signed URLs, records metadata, and answers queries, while every byte of video flows directly between the browser, Amazon S3, and CloudFront. This separation has three consequences that together justify the architecture. The API server's resource consumption becomes independent of video size and upload concurrency. The processing tier can scale according to transcoding demand without affecting the responsiveness of the interface. And a failure in the processing tier degrades the system gracefully, leaving

already-published videos playable while newly uploaded ones remain in a pending state.

4.2 Transcoding Pipeline Design

Figure 4.2 traces the sequence of interactions from the moment the user initiates an upload until the video becomes available.

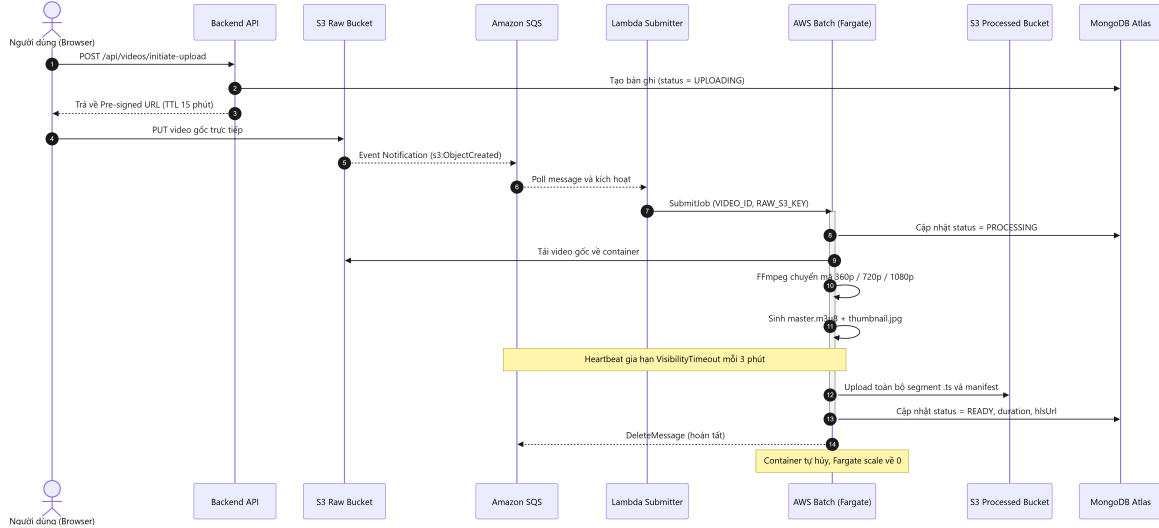


Figure 4.2: *Sequence diagram of the event-driven transcoding pipeline*

Two design decisions embedded in this sequence merit explanation.

The first concerns the ordering of database record creation and URL issuance. The backend creates the database record *before* generating the pre-signed URL, and derives the S3 object key from the identifier of that record. The alternative, generating a key first and creating the record afterwards, introduces a race condition: the S3 event notification can arrive and trigger transcoding before the record exists, at which point the transcoder has no row to update and the job fails. Creating the record first eliminates this class of failure entirely.

The second concerns the visibility timeout of the SQS message. Amazon SQS hides a message from other consumers for a configured period after it is received, and redelivers it if the consumer has not deleted it by then. Transcoding a large file can easily exceed any reasonable fixed timeout, which would cause the same video to be processed twice concurrently. The transcoder therefore implements the heartbeat pattern: while FFmpeg runs, a timer periodically extends the visibility timeout of the in-flight message. The message is deleted only after the job completes successfully. If the container crashes, the heartbeat stops, the timeout lapses, and the message returns to the queue for another attempt.

4.3 Database Design

The persistence layer uses MongoDB Atlas with three collections, shown in Figure 4.3.

USER			
ObjectId	_id	PK	
string	username	UK	
string	email	UK	
string	password		bcrypt hash, select false
string	displayName		
string	avatar		
string	channelDescription		
number	subscribers		
string	resetPasswordToken		
Date	resetPasswordExpire		
Date	createdAt		
Date	updatedAt		

tải lên

VIDEO			
ObjectId	_id	PK	
ObjectId	user	FK	
string	title		
string	description		
string	category		enum 7 danh mục
string	status		UPLOADING PROCESSING READY ERROR
string	visibility		public private unlisted
string	rawS3Key		
string	hlsUrl		CloudFront master m3u8
string	thumbnailUrl		
number	duration		giây
number	fileSize		bytes
string	mimeType		
number	views		
ObjectId_array	likes	FK	

viết

The `Video` document carries a `status` field that acts as a state machine with four values. A record begins as `UPLOADING` when the pre-signed URL is issued, moves to `PROCESSING` once the upload is confirmed and the job is submitted, and finally reaches either `READY` or `ERROR`. Modelling the lifecycle explicitly rather than inferring it from the presence of other fields makes the interface straightforward: the client simply renders a different state for each value, and a video that failed to transcode is distinguishable from one that is merely slow.

Likes and dislikes are stored as arrays of user identifiers embedded within the video document rather than as a separate collection. This denormalisation is appropriate here because the counts are always read together with the video and because a single user can appear at most once in each array, which bounds the array size by the number of viewers rather than allowing unbounded growth.

Indexes are defined to match the actual query patterns. A compound index on visibility, status, and creation time supports the home page listing, which always filters on the first two fields and sorts on the third. A text index on title and description supports the search feature.

4.4 Content Protection Design

Protecting private content requires two complementary mechanisms, because each addresses a different threat.

Origin Access Control makes the processed S3 bucket entirely private and grants read access exclusively to the CloudFront distribution through a bucket policy. This prevents a viewer from bypassing the content delivery network and fetching objects directly from S3, which would both circumvent any access control implemented at the edge and incur higher data transfer charges.

Origin Access Control alone, however, does not determine *who* may retrieve content through CloudFront. Without a second mechanism, any person holding a manifest URL can download the video regardless of its visibility setting, which would make the private mode ineffective in practice. The system therefore adds CloudFront Signed Cookies backed by a trusted key group [8]. The backend verifies the requester's permission using exactly the same service function that governs the API response, signs a policy scoped to the directory of that single video, and sets the resulting cookies on the response. CloudFront rejects any request lacking a valid signature with HTTP 403 at the edge.

Signed cookies were chosen over signed URLs because of the structure of HLS. A playback session retrieves a manifest and then hundreds of individual segment files, each as a separate request. Signing each URL individually would require rewriting every entry inside the manifest, typically using an edge function, which adds considerable complexity and cost. A single set of cookies covers the entire directory and is attached automatically

by the browser to every subsequent request.

The policy is signed as a custom policy rather than a canned policy. This distinction is not cosmetic: a canned policy does not permit wildcards in the resource path, whereas the required scope is an entire directory. The custom policy expresses the resource as a pattern covering the manifest and all segments of one specific video, which grants exactly the access needed and no more.

4.5 Interface Design

The frontend is a single page application composed of functional React components with hooks, using React Context for authentication and theme state. Redux was considered unnecessary at this scale, since the shared state consists only of the current user and the selected theme.

The video player is implemented as a custom component rather than relying on the browser's native controls. The reason is that native controls cannot be extended: the browser renders its own separate menu for playback speed and picture-in-picture, which cannot be merged with the quality selector that HLS.js exposes. Presenting the user with two unrelated menus would be confusing, so the component reimplements play and pause, seeking, volume, fullscreen, playback speed, picture-in-picture, and rendition selection within a single settings menu.

The player also determines when a view is counted. Rather than incrementing the counter when the page loads, which would allow the count to be inflated by repeated refreshes, the component notifies the page only after playback has passed a threshold of several seconds, and the server additionally ignores repeated reports from the same viewer within a thirty-minute window and never counts the owner's own views.

Chapter 5

Implementation and Experimental Results

5.1 Backend Implementation

The backend follows a layered structure in which routes delegate to controllers, controllers delegate to services, and only services interact with the data models. Business logic is kept out of route handlers so that it can be unit tested without an HTTP server.

Authorisation for video access is concentrated in a single service function. Every path that exposes a video, whether the detail endpoint, the like and comment endpoints, or the playback authorisation endpoint, calls that same function. This is a deliberate choice: duplicating the permission check across endpoints is the most common way for an access control gap to appear, because a later change updates one copy and not the others.

The upload flow validates metadata on the server before issuing a pre-signed URL. Client-side validation alone is insufficient, since a user can call the API directly and obtain a URL for an arbitrary file, effectively turning the project's storage bucket into free hosting for unrelated content. The server therefore checks the declared MIME type against an allow-list of video formats and rejects files exceeding two gigabytes.

Several protective measures were added at the HTTP layer. Security headers are applied through Helmet. Cross-origin requests are restricted to an explicit allow-list rather than being accepted from any origin, which is also a prerequisite for signed cookies to work, because the CORS specification forbids credentials in combination with a wildcard origin. Authentication endpoints are rate limited to ten attempts per fifteen minutes per address, which makes password guessing impractical without affecting legitimate users. Finally, the process validates its required environment variables at startup and refuses to start if the token signing secret is missing, converting a dangerous runtime failure into an immediate and obvious one.

5.2 Transcoder Implementation

The transcoder is a Node.js worker packaged with FFmpeg in a Docker image built through a multi-stage build. The base image went through two upgrades over the course of the project. Node 18 was used initially but had to be abandoned because Mongoose 9 depends on the Web Crypto API, which is not fully available in that version, causing the container to fail when connecting to MongoDB Atlas. Node 20 was then adopted, and later replaced with `node:24-slim` once Node 20 reached end-of-life on 30 April 2026 and stopped receiving security patches; Node 24 was chosen over the newer Node 26 because it is the current Active LTS release, supported until April 2028, whereas Node 26 had not yet entered its LTS phase at the time of the upgrade. The Alpine variant was avoided throughout because FFmpeg requires glibc.

The heartbeat mechanism described in Section 4.2 is, in effect, a lease in the sense introduced by Burrows for the Chubby lock service [9]: a holder retains a resource by periodically renewing it before a timeout expires, and Burrows already identified the classical failure mode of that pattern — a lease can expire while its holder is merely stalled rather than dead, so a second holder may be granted the same resource while the first is still alive and will eventually act on it. A defect matching exactly this failure mode surfaced during a literature-driven review of the pipeline’s reliability guarantees, discussed further in Section 5.5: neither `updateVideoReady` nor `updateVideoError` checked the video’s current status before writing, so a duplicate job triggered by an SQS redelivery could silently overwrite a successful result, and in the worst case could revert an already-`READY` video back to `ERROR`. Both functions were changed to a conditional `findOneAndUpdate` that only writes when the current status is not already `READY`, and `processVideo` now checks the video’s status before downloading anything, so a duplicate delivery that arrives after the first job has already finished is rejected immediately instead of re-downloading and re-transcoding the source.

A single FFmpeg invocation produces all three renditions, which is more efficient than three separate invocations because the source is decoded only once. The command emits segments of six seconds, generates the per-rendition playlists, and the worker then writes the master playlist that lists the available bandwidths and resolutions. A thumbnail is extracted in a separate pass.

Failure handling is deliberate. When FFmpeg exits with a non-zero status, the error propagates to the top-level handler, which records the `ERROR` status in the database and, crucially, does *not* delete the SQS message. Leaving the message in flight allows the visibility timeout to lapse so that the job is retried, and after the configured number of failed attempts the message is moved to the dead letter queue where it can be inspected rather than lost.

5.3 Infrastructure as Code

The AWS environment is described by eleven Terraform modules, summarised in Table 5.1, and is instantiated separately for the development and production environments.

Table 5.1: Terraform modules and their responsibilities

MODULE	RESPONSIBILITY
s3	Raw and processed buckets, CORS configuration, Glacier lifecycle transition, event notification
sqs	Transcode queue, dead letter queue with a redrive policy, queue access policy
ecr	Container registry with image scanning on push and a lifecycle policy retaining recent images
iam	Execution roles for AWS Batch, the ECS task, the transcoder task, and the Lambda submitter
vpc	Virtual network, public subnets, internet gateway, security groups, S3 gateway endpoint
secrets	AWS Secrets Manager entries for the database connection string and the token signing secret
sns	Notification topics for transcode completion and dead letter queue alerts
monitoring	CloudWatch log groups, alarms, and dashboard
batch	Fargate compute environment, job queue, and job definition
lambda	Job submitter function and its SQS event source mapping
cloudfront	Distribution, Origin Access Control, cache policy, and the trusted key group used for signed cookies

Two configuration details are worth highlighting. The S3 lifecycle policy transitions raw source files to Glacier after thirty days, since they are never read again once transcoding has succeeded but must be retained in case a re-encode becomes necessary. The cache policy assigns a long time-to-live to segments, which are immutable once produced, and a short one to manifests, which may change if the set of renditions changes.

5.4 CI/CD Pipeline and DevSecOps

Seven GitHub Actions workflows automate verification and deployment, as shown in Figure 5.1.

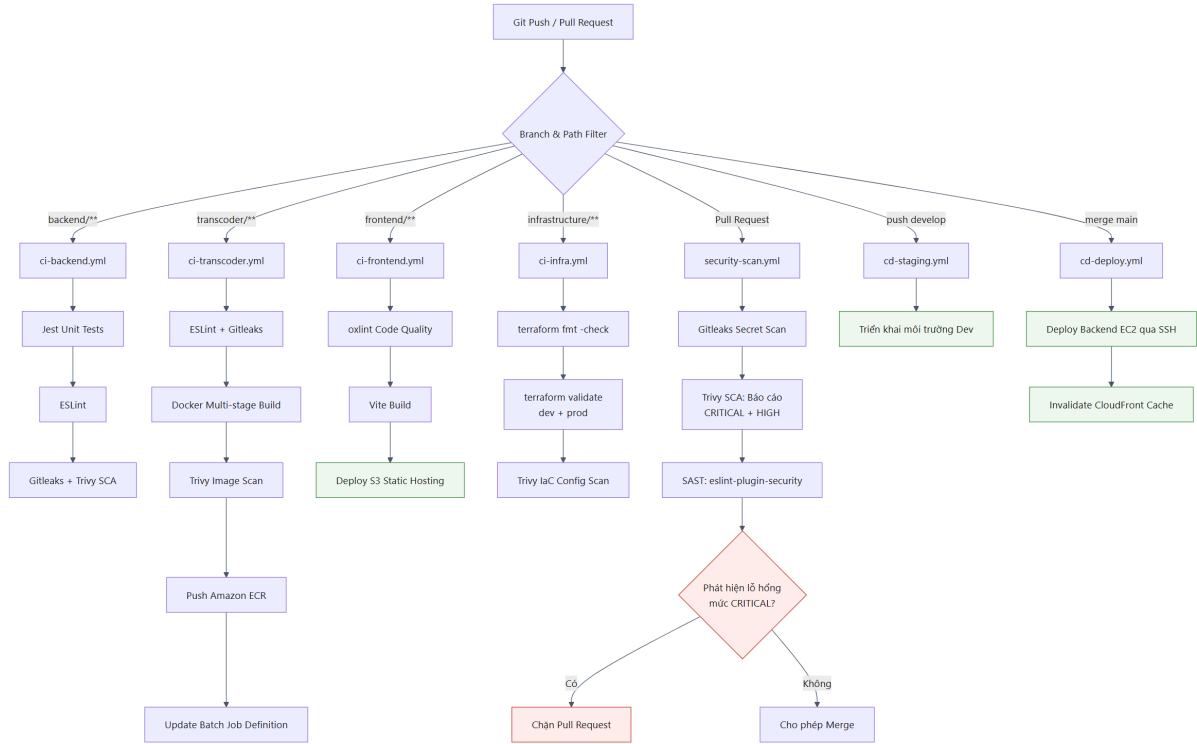


Figure 5.1: *CI/CD workflows and the two-tier DevSecOps quality gate*

The security gate is structured in two tiers, and the reason is practical. Scanning at the combined critical and high severity levels and failing the build on any finding sounds rigorous, but in practice most high-severity findings originate in transitive dependencies for which no patch yet exists. A gate configured that way fails continuously for reasons the team cannot act upon, and the predictable result is that developers begin ignoring it. The pipeline therefore reports critical and high findings for visibility without blocking, and blocks only on critical findings for which a fix is available. This keeps the gate meaningful, because a failure now always corresponds to an action the team can actually take.

Static application security testing is performed with `eslint-plugin-security` rather than SonarQube. The rule set detects patterns such as command execution built from user input, regular expressions vulnerable to catastrophic backtracking, and cryptographically unsafe random number generation. The choice avoids operating a separate analysis server while still executing genuinely within the pipeline. Where a finding is a false positive, it is suppressed with an inline annotation that documents why, which forces each suppression to be justified rather than silently ignored.

Infrastructure code is validated in its own workflow, which checks formatting, initialises without a remote backend so that no credentials are required, validates both environments, and scans the configuration for insecure settings.

5.5 Testing

The automated test suite comprises 98 tests across eight suites, summarised in Table 5.2.

Table 5.2: Automated test coverage by area

SUITE	TESTS	FOCUS
videoService	11	Upload initiation, ownership checks, deletion cascade
s3Service	2	Key generation and pre-signed URL creation
authService	17	Registration conflicts, login failures, token claims, password reset
authMiddleware	9	Missing, malformed, forged, and expired tokens
visibility	21	Privacy enforcement across every query path, view deduplication
uploadValidation	11	HTTP-level validation of file type and size
cloudfrontService	21	Cookie signing, policy scope, cookie attributes
transcoder/dbHandler	6	Conditional writes preventing duplicate-job overwrites

The visibility suite is the most important from a security standpoint, since it asserts that a private video is absent from the public listing, returns a not-found response rather than a forbidden one to non-owners, and remains accessible to its owner. Returning not-found is deliberate: a forbidden response would confirm that a video with that identifier exists, which leaks information.

The value of testing was demonstrated concretely during development of the signing service. An initial implementation produced a canned policy, which the test suite immediately flagged because the resulting cookie set contained an expiry field instead of a policy field. Since canned policies do not support wildcard resources, that implementation would have failed for every video once deployed. The defect was caught before any infrastructure was provisioned.

Testing caught a second defect in a similar way, this time surfaced not by writing a test first but by a literature review of the reliability guarantees the pipeline was implicitly assuming (Section 5.2). Once the conditional-write fix was implemented, six tests were written to pin down the exact contract: a normal write succeeds when the video is not yet **READY**; a write that loses the race to an earlier successful write is rejected and the function returns the winning record rather than throwing; and a late failure from a duplicate job can never revert an already-**READY** video to **ERROR**. As a falsifiability check, the same six tests were run against the original, unconditional implementation: four of the six failed, either because it never calls `findOneAndUpdate` at all or because it crashes outright when handed a plain record with no `save` method, which is exactly the shape a conditional read would have to return. The two that still passed were the not-found cases, which neither version distinguishes from a duplicate write. The result confirms both that the new tests

are actually exercising the fixed code path and that the original implementation could not have satisfied the contract by construction, not merely by accident of the test data chosen.

5.6 Quality of Experience Evaluation

Time-to-First-Frame is measured by an instrumentation script that fetches the master playlist, parses it to locate a rendition playlist, fetches that playlist, and finally downloads the first media segment, recording the elapsed time for each stage. The measurement is repeated so that mean, median, and 95th percentile values can be reported, because a single sample is meaningless when network latency varies and the first request is typically an edge cache miss.

Table 5.3: Time-to-First-Frame measurement results

STAGE	MEAN	MEDIAN	P95	MIN	MAX
Master playlist	95.55	82.49	355.02	76.61	355.02
Rendition playlist	88.62	82.40	207.32	77.62	207.32
First segment	108.76	87.04	322.20	78.41	322.20
Total TTFF	292.93	251.52	884.54	242.23	884.54

Note: values in milliseconds, from Table 5.3, $n = 20$ samples, populated from `docs/results/qoe-ttff.js` produced by `scripts/benchmark-qoe.js` against a video served directly from the S3 bucket (source URL recorded in the result file), since CloudFront was not yet provisioned at measurement time (Section 6.2) — these figures therefore represent origin latency without CDN edge caching, and would be expected to improve once CloudFront is enabled. The mean (292.93 ms) exceeds the median (251.52 ms) because one outlier sample (884.54 ms) inflates both the mean and the P95/max columns, which coincide with the single-sample maximum at this sample size.

5.7 Transcoding Pipeline Performance

Pipeline latency is measured from the completion of the upload to the moment the record reaches the `READY` state, which corresponds to the interval a user actually waits. This interval encompasses the S3 event delay, the time spent queued in SQS, container startup on Fargate, FFmpeg execution, and the upload of results.

Table 5.4: Transcoding pipeline latency by input size

SIZE	VIDEO DURA- TION	UPLOAD (S)	PIPELINE (S)	REALTIME FACTOR
100 ($n=3$)	MB 120 s	3.13	475.80	3.97
500 ($n=1$)	MB 600 s	13.13	2281.25	3.80
1 ($n=1$)	GB 1600 s	72.00	5917.03	3.70

Note: measured at the 1 vCPU / 2 GB job configuration, populated from `docs/results/transcode-time`. The vCPU allocation of every run in this section was confirmed after the fact by querying `aws batch describe-jobs` for the job definition revision each job actually executed against, rather than relying on the labels recorded by the measurement script; all rows in this table resolved to revision 4 (1 vCPU / 2 GB) and all rows in the comparison below to revision 5 (4 vCPU / 8 GB). Pipeline and upload columns are the arithmetic mean across replications. Realtime factor is pipeline time divided by video duration, so a value above 1.0 means transcoding took longer than the video’s own runtime; the three sizes fall in a narrow band of $3.70\text{--}3.97\times$ real time at 1 vCPU, which indicates the pipeline scales essentially linearly with content duration at fixed compute. The 500 MB and 1 GB rows have only a single replication each because a full run at these sizes takes 30–100 minutes of AWS Batch time, which made repeated sampling costly within the project’s time and budget; this is a threat to the precision of those two rows that Chapter 6 returns to. For the 100 MB size, the three individual runs (346.32 s, 539.80 s, 541.28 s) did not show the monotonic cold-start slowdown the measurement protocol anticipated — the first run was in fact the fastest of the three — which suggests that Fargate Spot capacity variability dominates over container image pull cost at this scale, rather than a first-run-specific effect.

A separate experiment raised the container allocation from 1 vCPU / 2 GB to 4 vCPU / 8 GB and repeated the measurement on the same two inputs. Pipeline time fell from a mean of 475.80 s to a mean of 115.56 s for the 100 MB input (3 runs each), and from 5917.03 s (1 run) to a mean of 1078.70 s for the 1 GB input (2 runs: 1080.29 s and 1077.10 s). Taken at face value these are speedups of $4.12\times$ and $5.49\times$ respectively.

Both figures exceed the $4\times$ that the change in vCPU count alone could account for, and that observation deserves scepticism rather than celebration. Amdahl’s Law bounds the achievable speedup *below* the resource ratio, because the serial portion of an encoder — entropy coding and rate control among others — does not parallelise; the multi-threaded H.264 literature reports exactly this sub-linear pattern, with Sankaraiah *et al.* measuring $1.97\times/3.96\times/7.71\times$ at 2/4/8 threads [10] and Chen *et al.* $4.7\text{--}8.6\times$ across six Cell

SPEs [11]. A measured result above the resource ratio therefore indicates that something other than thread-level parallelism is contributing, and two confounds are visible in the data.

The first is that the experiment did not vary vCPU count in isolation: memory was raised from 2 GB to 8 GB at the same time, so any benefit from reduced memory pressure during the concurrent encoding of three renditions is folded into the same measurement. The second, and more serious, is the instability of the 1 vCPU baseline. The three 100 MB runs at 1 vCPU span 346.32 s to 541.28 s, a spread of 56.3%, whereas the corresponding 4 vCPU runs span only 24.0%. The speedup estimate is consequently very sensitive to which baseline statistic is chosen: $3.29\times$ comparing the fastest run of each configuration, $4.12\times$ comparing means, $4.15\times$ comparing slowest runs, and $4.86\times$ comparing medians. Only the first of these is consistent with the sub-linear bound the theory predicts. The 1 GB comparison rests on a single 1 vCPU observation and so admits no variance estimate at all.

The honest conclusion is therefore narrower than the headline numbers suggest: raising the allocation from 1 to 4 vCPU reduced pipeline latency by roughly a factor of four on this workload, but the present sample is too small and too noisy to establish whether the true scaling is sub-linear as theory requires, and the simultaneous memory change means the experiment does not isolate the effect of vCPU count. Establishing the scaling behaviour properly would require repeated 1 vCPU baselines — the variance there, not the 4 vCPU measurements, is what limits the conclusion — and a separate arm that varies memory alone. This is recorded as future work in Section 6.3 rather than presented as a settled result.

Cost per video, by contrast, is robust to this uncertainty: AWS Batch on Fargate bills per vCPU-second, so roughly four times the hourly rate for roughly one quarter of the duration leaves the total charge per video essentially unchanged. The practical implication is that the 4 vCPU configuration buys a substantially lower waiting time for the user at close to no additional cost, which holds regardless of whether the exact speedup is $3.3\times$ or $4.9\times$.

Note: populated from docs/results/transcode-timing.json. Each size should be measured at least three times, and the first run reported separately, because it includes the cost of pulling the container image and is therefore substantially slower than the steady state.

5.8 Load Testing

Scalability is assessed by submitting concurrent uploads with a virtual user count ramping to one hundred, verifying that the queue retains all work and that AWS Batch scales to process it. Upload payloads in this test are synthetic (a small fixed byte string rather than a real video file), because the object here is to exercise the upload API, S3 event

notification, SQS queue, and Batch job submission path under concurrent load, not to measure transcoding quality; every submitted job is consequently expected to fail at the FFmpeg stage on invalid input, and job outcome is not the metric of interest — queue behaviour is.

Table 5.5: Concurrent upload stress test results (API layer)

METRIC	VALUE
Total requests	100
Throughput (requests/s)	15.52
Failure rate	0%
Mean response time (ms)	2388.52
95th percentile (ms)	3317.79
Peak virtual users	50

Note: populated from `docs/results/node-load-test.json`, produced by `scripts/node-load-test.js` a native-fetch Node.js load generator. All 100 uploads (initiate, direct-to-S3 PUT, confirm) succeeded at the API layer.

5.8.1 Cross-Validation with an Independent Tool

A single load generator can produce a misleading result if the generator itself, rather than the system under test, is the bottleneck. The experiment was therefore repeated with k6, an industry-standard load testing tool implemented in Go, driving the same three-step upload sequence against the same deployed endpoint at the same concurrency (50 virtual users, 100 total iterations). The k6 scenario uses the `shared-iterations` executor rather than the time-based ramp that the script originally specified, so that the total unit of work is fixed at exactly 100 uploads and is therefore directly comparable rather than merely similar.

Table 5.6: Cross-validation of the load test across two independent tools

METRIC	NODE.JS	K6
Upload cycles completed	100	100
Failure rate	0%	0%
Batch jobs queued (RUNNABLE)	92	92
Batch jobs admitted (STARTING)	8	8
SQS messages remaining	0	0
Queue drain, 76 jobs (s)	416	417
Consumption rate (jobs/min)	10.96	10.94

The two tools agree exactly on every infrastructure-level observation. Immediately after each burst the queue held 92 jobs waiting with 8 admitted for execution, and Amazon

SQS reported zero messages outstanding. The drain rate agrees to within 0.24%: measuring from the common reference point at which 76 jobs remained queued until the queue emptied took 416 s in the Node.js run and 417 s in the k6 run, a consumption rate of 10.96 against 10.94 jobs per minute. Reproducing both the 92/8 split and the drain rate across two independently implemented generators running on different runtimes is considerably stronger evidence than either run alone that the eight-job ceiling and the resulting throughput are properties of the compute environment's configuration — `max_vcpus = 8` divided by `job_vcpu = 1` — rather than artefacts of how one particular client paced its requests.

The latency figures, by contrast, are *not* directly comparable, and reporting them side by side without qualification would be an error. The Node.js script times each complete upload cycle, so its mean of 2388.52 ms covers all three HTTP requests together; k6's `http_req_duration` times each HTTP request individually, giving a mean of 670.03 ms per request across 300 requests. Scaling the latter by the three requests per cycle yields 2010.09 ms, within 15.8% of the Node.js figure — close enough to corroborate it, and the residual gap is consistent with k6 running inside a Docker container with an additional network hop that the natively executed Node.js script did not incur. Throughput agrees on the same basis once k6's virtual-user initialisation is accounted for: the raw 100 cycles over 9.83 s understates the steady-state rate, because no iteration completed during the first four seconds, whereas the 100 cycles that completed in the remaining 5.8 s correspond to 17.24 cycles per second against the Node.js script's 15.53.

The queue and compute layer were observed separately by polling the AWS Batch job queue every 25 seconds after the 100 uploads completed. Two data points anchor this observation: immediately after the burst, 92 jobs were queued in the `RUNNABLE` state with 8 already `STARTING`, and the Amazon SQS queue itself reported zero messages remaining, meaning the Lambda job submitter had already drained every event into AWS Batch within seconds of the S3 notifications firing. The `STARTING` count held at 7–8 throughout the drain, matching `max_vcpus = 8` divided by `job_vcpu = 1` exactly, which confirms the compute environment enforced its configured concurrency ceiling rather than either exceeding it or under-utilising it. The backlog of 92 queued jobs reached zero 456 seconds after the burst, and no message was lost or left stuck; the full sample sequence is recorded in `docs/results/stress-test-concurrency.json`. This is the direct evidence for the claim that opened this section: Amazon SQS retained every unit of work submitted faster than the compute layer could consume it, and AWS Batch scaled out to, and stayed within, its configured limit while consuming that backlog.

5.9 Cost Analysis

The cost comparison contrasts a traditional deployment, in which an Amazon EC2 instance runs FFmpeg continuously, against the serverless container architecture. The workload assumption is three hundred videos per month averaging five minutes each, with a mean transcoding time of forty-five seconds, giving 3.75 hours of actual processing per month.

Table 5.7: Monthly operating cost comparison (ap-southeast-1)

COMPONENT	EC2 (ALWAYS ON)	SERVERLESS	SAVING
Compute	c5.xlarge for 730 h: \$124.10	Fargate Spot for 3.75 h: \$0.05	99.96%
Storage	EBS 100 GB plus S3 50 GB: \$9.25	S3 with Glacier lifecycle: \$1.45	84.32%
Queueing	Self-hosted broker	SQS and Lambda: \$0.01	—
Total	\$133.35	\$1.51	98.87%

Content delivery cost is excluded from Table 5.7 because it is neutral with respect to the architectural choice: the volume of data served to viewers is identical whichever compute model produces the renditions. Attributing a CloudFront charge to the EC2 column while crediting the serverless column with the free tier would overstate the saving. When delivery is included on both sides at post-free-tier rates, the total becomes \$145.35 against \$13.51, a saving of approximately 90.7%, which is the more conservative figure and the one that should be quoted for total cost of ownership at scale.

The saving originates entirely in the elimination of idle capacity. Under the stated assumptions the always-on instance is billed for 730 hours while performing useful work for only 3.75 of them, a utilisation of roughly 0.5%. It follows that the advantage is not universal: as sustained utilisation rises, per-second Fargate billing approaches and eventually exceeds the cost of a reserved instance. The architecture is therefore best suited to workloads that are genuinely bursty, which is the case for user-generated video uploads.

Chapter 6

Conclusion and Future Work

6.1 Summary of Achievements

This project set out to build a video sharing platform whose transcoding subsystem runs on a serverless container and event-driven architecture, and to evaluate whether that choice delivers the cost and scalability benefits it promises.

The functional platform was completed. Users can register, authenticate, upload videos directly to Amazon S3 through pre-signed URLs, watch them with adaptive bitrate playback across three renditions, control whether each video is public or private, manage their channel, search and filter the catalogue, and interact through likes and comments.

The transcoding pipeline operates without manual intervention. An upload emits an S3 event that is buffered in Amazon SQS and dispatched by an AWS Lambda function to AWS Batch, where a container packaging FFmpeg produces the renditions, the master playlist, and a thumbnail before terminating. The heartbeat pattern prevents duplicate processing of long jobs, and failed jobs are retried and ultimately isolated in a dead letter queue rather than being lost silently.

Content protection was implemented in two layers. Origin Access Control keeps the storage bucket entirely private, and CloudFront Signed Cookies ensure that private videos are rejected at the edge even when the exact manifest URL is known. This closes a gap that would otherwise have made the private visibility setting ineffective in practice, since the API can restrict its own responses but cannot restrict a content delivery network.

The infrastructure is fully described as code in eleven Terraform modules across two environments, and seven GitHub Actions workflows automate linting, unit testing, static security analysis, container scanning, infrastructure validation, and deployment. The security gate blocks merges on fixable critical findings rather than merely reporting them.

The automated test suite grew to 92 tests. Its practical value was demonstrated when it caught a defect in the cookie signing implementation that would have prevented every private video from playing once deployed.

6.2 Limitations

Several limitations should be stated plainly.

The bitrate ladder is fixed at three rungs regardless of content. As the literature reviewed in Section 2.2 shows, the optimal ladder is content-dependent, and a fixed ladder therefore wastes bandwidth on simple content while under-serving complex content. Netflix reported bitrate reductions between 28% and 38% at equivalent perceptual quality by optimising the ladder per shot [7], which indicates the magnitude of the opportunity forgone.

Each video is transcoded by a single container, without the chunk-based parallelism that Netflix employs. Processing time consequently scales linearly with video duration, and a long video cannot be accelerated by adding capacity. This is acceptable at the scale of this project but would become the dominant constraint for longer content.

Signed cookies remain valid for two hours after issuance. If an owner switches a video from public to private, viewers holding unexpired cookies retain access until those cookies lapse. Immediate revocation would require rotating the signing key or introducing an edge function that performs a per-request check.

A more fundamental limitation surfaced during the preparation of the experimental results for this report. Verifying the deployed environment directly through the AWS command line interface revealed that no CloudFront distribution had ever been created in the development AWS account, and that the processed content bucket carried a manually added policy granting anonymous `s3:GetObject` access to any object. In other words, the content protection scheme described in Section 4.4 was fully implemented and validated in code but had never actually been deployed; every video, public or private, was directly retrievable from Amazon S3 without any authorisation check. Attempting to apply the Terraform configuration to correct this failed with an access-denied error, because AWS requires manual account verification before a new account may create CloudFront distributions, a restriction the project had encountered before and for which the `enable_cloudfront` flag had originally been introduced as a workaround. A support request was filed with AWS to lift this restriction; at the time of writing, several days later, it remains pending, and repeated attempts to re-apply the Terraform configuration continue to return the identical access-denied error, confirming the restriction is still in effect rather than merely slow to propagate. This episode is reported here in the interest of honesty rather than omitted, and it illustrates a broader lesson about serverless and managed cloud services: correctness of the Infrastructure as Code is necessary but not sufficient, since provisioning can still be blocked by account-level policy outside the application's control, and verifying claims against the live environment rather than against the source code alone is essential before a security control can be considered in effect.

Because the wait for account verification could not be bounded, a temporary substitute

was put in place so the deployed system would not be left serving plain HTTP indefinitely. Cloudflare’s free tier was configured as an interim DNS and TLS-terminating proxy in front of the same S3-hosted frontend, and a small Nginx reverse proxy was added on the backend’s EC2 instance so the API could also be reached over HTTPS, since Cloudflare’s free tier only forwards a fixed set of ports and the API’s port was not among them. This restores end-to-end HTTPS for users immediately, but it is explicitly a stopgap rather than a substitute for the evaluated design: it protects the connection in transit, not the content itself, so the manually added public bucket policy described above still leaves every object directly fetchable from S3 by anyone who obtains its URL, and the CloudFront Signed Cookie mechanism evaluated in Section 4.4 remains unenforced until the real CloudFront distribution can be created. The Terraform configuration for that distribution needs no changes once AWS lifts the restriction; the only remaining step is to re-point DNS from the interim S3 target to the CloudFront domain name, which is expected to take minutes rather than days.

The system implements no digital rights management. An authorised viewer can capture the decrypted stream or share their cookies, and preventing this is outside the scope of the techniques used here.

Finally, the rate limiting and view deduplication mechanisms hold their state in process memory. This is correct for the current single-instance deployment but would need to move to a shared store such as Redis before the backend could be scaled horizontally, since each instance would otherwise enforce its own independent limits.

6.3 Future Work

The most valuable extension would be per-title bitrate ladder optimisation driven by a perceptual quality metric such as VMAF. Encoding a short probe of the source at several operating points and selecting those on the convex hull of the quality-versus-bitrate curve would reduce delivery bandwidth at unchanged perceived quality, which lowers the largest remaining cost component of the system.

Chunk-based parallel transcoding would address the linear scaling of processing time. Splitting the source at keyframe boundaries, encoding the segments concurrently across multiple containers, and concatenating the results would convert additional capacity into reduced latency. The main challenge is maintaining rate control consistency across chunk boundaries so that quality does not fluctuate visibly.

Hardware acceleration offers a further avenue. Research on GPU-enabled serverless platforms for HEVC encoding [12] suggests substantial throughput gains, and adopting a more efficient codec such as HEVC or AV1 alongside H.264 would reduce bandwidth for clients that support it.

Finally, moving the rate limiting and deduplication state into a shared store, and

adding distributed tracing across the S3, SQS, Lambda, and Batch boundaries, would prepare the system for horizontal scaling and make failures in the asynchronous pipeline substantially easier to diagnose.

Bibliography

- [1] Netflix Technology Blog, “Rebuilding netflix video processing pipeline with microservices.” <https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359>, 2024. Accessed: 2026-08-10.
- [2] Netflix Technology Blog, “The making of VES: the cosmos microservice for netflix video encoding.” <https://netflixtechblog.com/the-making-of-ves-the-cosmos-microservice-for-netflix-video-encoding-946b9b3cd300>, 2024. Accessed: 2026-08-10.
- [3] Amazon Web Services, “Video on demand on AWS — implementation guide.” <https://docs.aws.amazon.com/solutions/latest/video-on-demand-on-aws/>, 2025. Accessed: 2026-08-10.
- [4] M. Zhang, Y. Zhu, C. Zhang, and J. Liu, “Video processing with serverless computing: A measurement study,” in *Proceedings of the 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2019.
- [5] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, “Cold start latency in serverless computing: A systematic review, taxonomy, and future directions,” *ACM Computing Surveys*, vol. 57, no. 3, 2025. Published version of arXiv:2310.08437.
- [6] Netflix Technology Blog, “Per-title encode optimization.” <https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2>, 2015. Accessed: 2026-08-10.
- [7] Netflix Technology Blog, “Dynamic optimizer — a perceptual video encoding optimization framework.” <https://netflixtechblog.com/dynamic-optimizer-a-perceptual-video-encoding-optimization-framework-e19f1e3a277f>, 2018. Accessed: 2026-08-10.
- [8] Amazon Web Services, “Serve private content with signed URLs and signed cookies — amazon cloudfront developer guide.” <https://docs.aws.amazon.com/>

`AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html`, 2025. Accessed: 2026-08-10.

- [9] M. Burrows, “The chubby lock service for loosely-coupled distributed systems,” in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [10] S. Sankaraiah, L. H. Shuan, C. Eswaran, and J. Abdullah, “Scalable video encoding with macroblock-level parallelism,” *EURASIP Journal on Advances in Signal Processing*, vol. 2014, p. 145, 2014.
- [11] C.-Y. Chen, S.-C. Cheng, *et al.*, “Task-based parallel H.264 video encoding for explicit communication architectures,” in *Proceedings of the International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*, 2011.
- [12] A. Salcedo-Navarro, R. Peña-Ortiz, J. M. Claver, M. Garcia-Pineda, and J. Gutierrez-Aguado, “Towards GPU-enabled serverless cloud edge platforms for accelerating HEVC video coding,” *Cluster Computing*, vol. 28, p. 68, 2025.

Appendix A

Supplementary Material

A.1 Repository Structure

The source code is organised as a monorepo with the following top-level layout.

```
DACNTT_VideoStreaming/  
  frontend/          React single page application (Vite, HLS.js)  
  backend/           Node.js Express API and test suite  
  transcoder/        FFmpeg worker and Docker image  
  infrastructure/  
    modules/         Eleven reusable Terraform modules  
    environments/    dev and prod instantiations  
  .github/workflows/ Seven CI/CD workflows  
  scripts/           Benchmarking and load testing scripts  
  docs/              Architecture diagrams, cost analysis, measured  
    results  
  report/            LaTeX sources of this report
```

A.2 FFmpeg Transcoding Command

The following invocation produces all three renditions in a single pass, decoding the source only once. The parameters shown are abbreviated for readability.

```
ffmpeg -i input.mp4 \  
  -filter_complex "[0:v]split=3[v1][v2][v3];[v1]\  
  [v1]scale=w=640:h=360[v1out];[v2]\  
  [v2]scale=w=1280:h=720[v2out];[v3]\  
  [v3]scale=w=1920:h=1080[v3out]" \  
  -map "[v1out]" -c:v:0 libx264 -b:v:0 400k \  
  -map "[v2out]" -c:v:1 libx264 -b:v:1 1500k \  
  -map "[v3out]" -c:v:2 libx264 -b:v:2 4000k \  
  -
```

```

-map a:0 -map a:0 -map a:0 -c:a aac \
-f hls -hls_time 6 -hls_playlist_type vod \
-master_pl_name master.m3u8 \
-var_stream_map "v:0,a:0_v:1,a:1_v:2,a:2" \
output/stream_%v/playlist.m3u8

```

A.3 Signed Cookie Custom Policy

The policy signed by the backend before issuing playback cookies. The wildcard in the resource path covers the manifest and every segment of one specific video, and nothing beyond it.

```

{
  "Statement": [
    {
      "Resource": "https://cdn.zelostech.site/videos/*/<videoId>/*",
      "Condition": {
        "DateLessThan": { "AWS:EpochTime": 1786300347 }
      }
    }
  ]
}

```

A.4 Reproducing the Measurements

The experimental results reported in Chapter 5 are produced by the scripts listed below. Detailed instructions, including the environment variables required and the order in which the measurements must be taken, are given in `docs/results/README.md` within the repository.

Table A.1: Measurement scripts and their output artefacts

SCRIPT	OUTPUT
<code>scripts/measure-transcode.js</code>	<code>docs/results/transcode-timing.json</code>
<code>scripts/benchmark-qoe.js</code>	<code>docs/results/qoe-ttff.json</code>
<code>scripts/k6-load-test.js</code>	<code>docs/results/k6-summary.json</code>
<code>scripts/node-load-test.js</code>	<code>docs/results/node-load-test.json</code>

A.5 Deploying the Infrastructure

The complete AWS environment is provisioned from a single directory.


```
cd infrastructure/environments/dev
terraform init
terraform apply
```

Enabling signed cookies additionally requires generating an RSA key pair, supplying the public key to Terraform, and configuring the backend with the corresponding private key and the key pair identifier reported by `terraform output`. The full procedure is documented in `docs/PRIVATE_VIDEO_SIGNED_COOKIES.md`.